LAUNCHING AI NATIVE VENTURES · SESSION 6 HACKATHON

One commitment. *One nudge.* One recorded outcome.

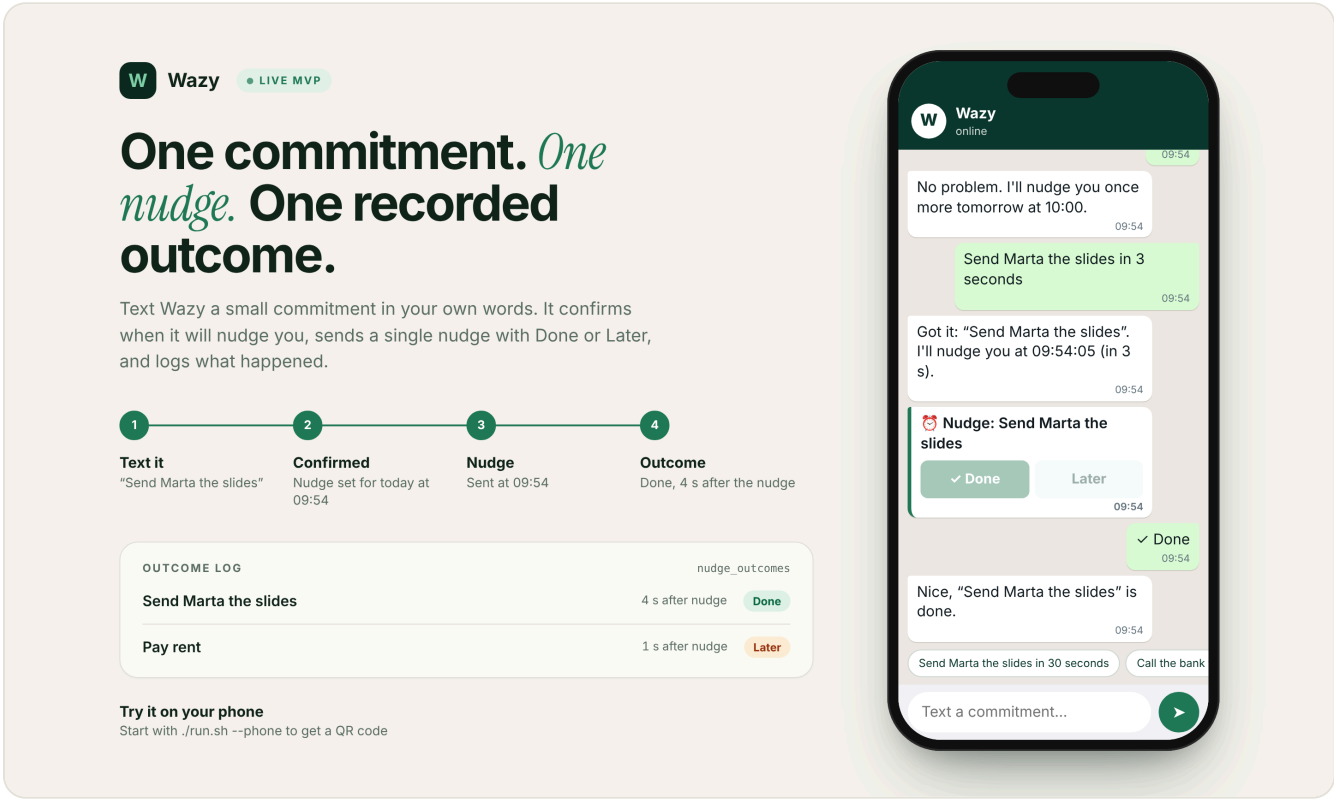
The working slice of Wazy we built and demoed live in class, now cleaned up into a package anyone can run on a laptop in one command. It tests one thing: whether a single, well-timed nudge helps busy people keep small promises without feeling nagged.

TEAM
Andrea Di Segni
David García

COURSE
Launching AI Native Ventures, ESADE

DATE
13 October 2026

CODE
`wazy-mvp/`
Python 3.9+, no installs



The presenter view after a run: the chat in a phone frame, the four-step tracker following the latest commitment, and the outcome log read straight from `nudge_outcomes`.

./run.sh

Starts everything. Phone access with `--phone`.

22 tests

Reading times, and the whole loop including Later, STOP and no reply.

No secrets

Nothing is needed to run it. Keys only ever go in a git-ignored `.env`.

~800 lines

of Python, standard library only. Every line can be explained.

1 · WHAT WE SET OUT TO TEST

Our riskiest guess, and the one goal for the sprint

> We believe that **one proactive nudge per commitment** will get busy people to follow through on small promises without feeling nagged. We will know we are right when test users text new commitments on **3+ different days** and tap **Done** on at least **half** of their nudges.

The one goal

A user texts a commitment, receives one nudge at the right time and taps Done or Later, with the outcome recorded. One build gives us two things: the measure for Guess 1, and the first step of the data-cycle defence from Session 5.

Why this and nothing else

Our old prototype (about 28,000 lines, March to April 2026) already does far more than anyone has asked for. The sprint's job was the thinnest loop, end to end.

What shipped

- ✓ **Text a commitment** in your own words (EN / ES).
- ✓ **One-line confirmation** with the exact nudge time.
- ✓ **One nudge** at that time: never early, never twice.
- ✓ **Done / Later**; Later nudges once more at 10:00 next day.
- ✓ **Every outcome recorded**, incl. STOP and no reply.
- ✓ **On a real phone**, via a tunnel and an access code.
- ✗ **Not WhatsApp yet:** a WhatsApp-style web chat (see 5).

2 · HOW A CONVERSATION GOES

Four steps, one screen

01
Text it
Send Marta the slides by Thursday

02
Confirmed
Got it: “Send Marta the slides”. I'll nudge you Thursday at 10:00.

03
One nudge
⏰ Nudge: Send Marta the slides
Done | Later

04
Outcome logged
One row in `nudge_outcomes`: done, later, stop or no reply, with the seconds it took.

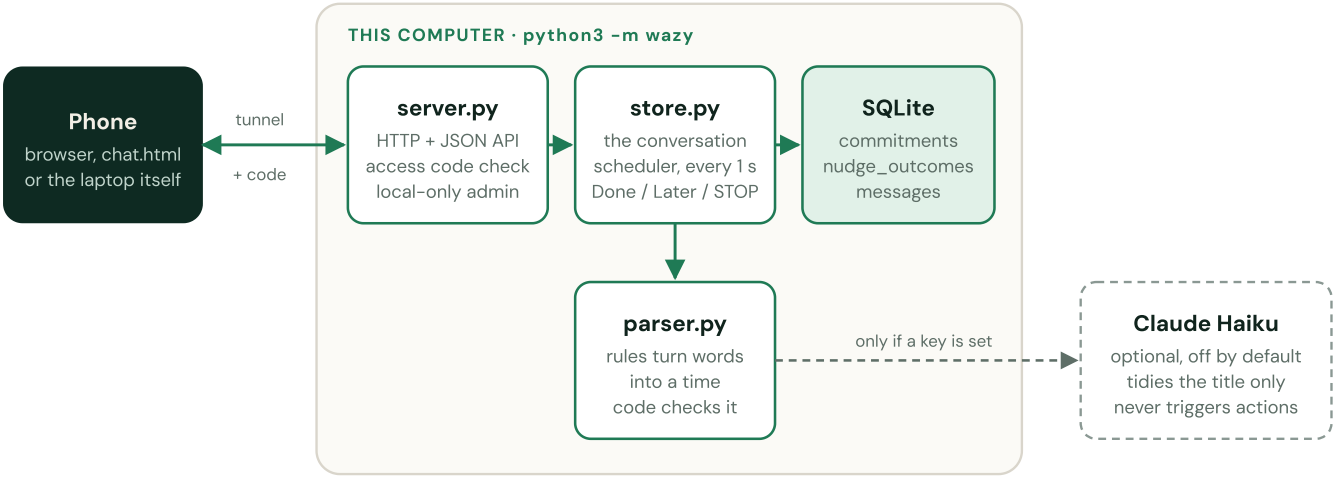
What the person sees

A chat that looks and behaves like WhatsApp: no app to install, no account, nothing to learn. The first message says what Wazy does, where the texts go, and that STOP ends everything. If no time is given, Wazy asks for one.

THEY TYPE	WAZY NUDGES
Pay rent in 2 minutes	in 2 minutes
Reply to Carlos by Thursday	Thursday 10:00
Call the bank tomorrow 9am	tomorrow 09:00
Llamar a mamá mañana a las 9	tomorrow 09:00
Buy milk	asks “When should I nudge you?”

3 · HOW IT IS BUILT

A small server on the laptop. Code decides when.



Three design rules

1. **Code decides when to nudge.** In Session 3 an AI checking dates added errors and doubled the cost, so the rules set the time. The AI is optional, only tidies the wording, and its answer can never trigger an action, which also blunts prompt injection.
2. **One commitment, one nudge, one recorded outcome.** The scheduler sends a due nudge exactly once and closes it after 24 hours if nobody answers.
3. **Fail closed.** A setting that is present but wrong (a short access code, a malformed key, a bad port) stops the server from starting.

The data cycle starts here

Every nudge ends in exactly one row of `nudge_outcomes`. Over weeks, those rows tell us when each person actually acts and which nudges they ignore, so Wazy can nudge less and at better times. That history is our defence: a copy of the code starts with none of it.

`done` `later` `stop` `no_reply` `pending`

The data model

TABLE	ONE ROW PER	KEY COLUMNS
commitments	thing the person promised	`raw_text, title, nudge_at, status, reader`
nudge_outcomes	nudge sent (the Guess 1 measure)	`nudged_at, outcome, responded_at, seconds_to_respond`
messages	line in the chat	`role, text, kind, commitment_id`

4 · RUN IT

Ready to run in one command

On a Mac (or any computer with Python 3.9+)

```
# 1. unzip and enter the folder
cd wazy-mvp

# 2. start it (no installs, no keys)
./run.sh

# 3. open the presenter view
open http://127.0.0.1:8765/demo
```

On a phone

```
# needs ngrok installed once
./run.sh --phone
# then scan the QR code on /demo
```

Tests

```
python3 -m unittest discover tests
# Ran 22 tests ... OK
```

What is in the folder

FILE	ITS ONE JOB
`wazy/config.py`	Settings from the environment or `.env`; refuses bad values.
`wazy/parser.py`	Turns the person's words into a title and a time.
`wazy/store.py`	Tables, the conversation, the scheduler.
`wazy/server.py`	Web server, access rules, JSON API.
`wazy/static/`	The chat (phone) and the presenter view.
`tests/`	22 tests: time reading and the full loop.
`architecturedoc.md` `scope.md`	Course documents, with the Session 6 log.
`run.sh, README.md`	Start script and instructions.

Settings (all optional, in `.env`)

VARIABLE	EFFECT
`ANTHROPIC_API_KEY`	Switches on Claude Haiku for the wording.
`WAZY_LATER_MINUTES`	Shorter “Later” for a live demo (default: next morning 10:00).
`WAZY_ACCESS_CODE`	Fixed phone code (generated if empty).
`WAZY_PORT, WAZY_DB`	Port (8765) and database file.

Demo script (90 seconds)

Type **“Send Marta the slides in 30 seconds”** → point at the confirmation and its exact time → while the tracker counts down, explain that code decides when and the AI only reads → the nudge arrives (the phone vibrates) → tap **Done** → point at the new row in the outcome log and its response time. Shift + R clears the demo.

What we checked, what we cut, and why

Security check

RISK	WHAT THE PACKAGE DOES
Leaked keys	No key is needed or included. The old prototype's `.env` is not part of this package; its Twilio and Anthropic keys were already rejected on 13 Oct and must be rotated before reuse.
Strangers reaching the laptop	Anything that comes through the tunnel needs the access code (QR link once, then an HttpOnly cookie). The presenter view, the tunnel link and reset only answer this computer.
Cross-site requests	The API only accepts JSON and checks the Origin header. Pages send a Content-Security-Policy and refuse to be framed by other sites.
Abuse and cost	Messages are cut to 300 characters, 30 a minute. The AI is off unless a key is set; one call per message, no retries.
Prompt injection	The AI's answer is only a title, shown as plain text, and a time that code validates. It cannot delete, send or change anything.
Privacy	The first message says where texts go and that STOP ends everything. Data stays in a local SQLite file; message text is never printed to the terminal.

What changed from the plan

The plan was to switch off 80% of the old Flask app with an `MVP_MODE` setting and run it on WhatsApp. At the start of the sprint the old Twilio and Anthropic keys were rejected (401). Rotating them and re-wiring the WhatsApp webhook did not fit the time box, so we built the loop as a separate, dependency-free slice and showed it as a WhatsApp-style web chat.

Cut to protect the goal

- WhatsApp and Twilio templates
- A phone-number allow-list (replaced by an access code)
- The AI reader switched on by default
- One link per tester, and an always-on host
- Everything in the scope sheet's “not now” list: calendars, summaries, recurring events, voice notes, payments

Honest limits today

- **Not WhatsApp yet**, so it does not yet test Guess 1 where people actually live.
- **One conversation per server**: one tester at a time.
- Nudges go out **only while the laptop is awake**.
- The rules understand the phrasings we wrote; the AI reader covers the rest when switched on.
- **No outside users yet.** Everything above was run by us.

Before the user test (Leon, Filip, Samuele)

1. Rotate the Twilio and Anthropic keys; set spending caps.
2. Connect the WhatsApp webhook and a Done / Later template to the same store.
3. One allow-listed number per tester.
4. Move to an always-on host.
5. Tell testers who processes their texts.

AI use: the code, tests and this document were produced with Claude under our direction; our AI Orchestration Log (Entry 6) records the prompts and what we changed or rejected. `architecturedoc.md` and `scope.md` sit next to this file.